République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research

École Nationale Supérieure d'Informatique (ESI) — Alger

Cycle Ingénieur — Spécialité Informatique — Projet Intégrateur IoT / Robotique

EsiBot

Robot Mobile Différentiel Autonome

Rapport Technique
Conception, Intégration et Évaluation Expérimentale
d'un Robot Mobile Autonome sous ROS 2 Humble

Module : Internet of Things — Robotique Mobile

Année Universitaire : 2025–2026

Chef d'équipe :
BOUMISSA Mohamed Chakib

Membres :
ALLOUCHE Reda
BENDAOUD Anes
CHABANE Ouail
IDJA Rayane
MOSTEFAI Mounir Sofiane
TATI Youcef

Résumé

EsiBot est un robot mobile autonome à entraînement différentiel conçu et réalisé par l'équipe dans le cadre du projet intégrateur IoT de l'École Nationale Supérieure d'Informatique (ESI Alger). Le cahier des charges initial s'inspire du TurtleBot3 Burger (ROBOTIS) : un ESP32-CAM devait piloter directement les moteurs, le servo-moteur et un radar ultrasonique rotatif (HC-SR04 + SG90), pendant que le Raspberry Pi 4 hébergeait la pile logicielle ROS 2 2 (Simultaneous Localization and Mapping (SLAM), Robot Operating System (ROS) 2 Nav2, tableau de bord web).

Au fil du développement, l'architecture matérielle réellement implémentée a significativement évolué par rapport à ce cahier des charges. Le pilotage des deux moteurs à courant continu, la lecture des encodeurs et l'asservissement de vitesse (Proportional-Integral-Derivative (PID)) sont assurés *directement* par les broches General-Purpose Input/Output (GPIO) du Raspberry Pi 4, épaulés par une centrale inertielle MPU6050 fusionnée par filtre de Kalman étendu (`robot_localization`). L'ESP32-CAM est conservé uniquement comme caméra de streaming Wireless Fidelity (IEEE 802.11) (WiFi) (Motion JPEG (MJPEG)), sans plus participer à la commande des actionneurs. Le radar ultrasonique rotatif a été implémenté en code mais n'est plus utilisé en exploitation courante : il a été remplacé par un capteur de profondeur Kinect (v1, ère Xbox 360), piloté par le pilote communautaire `kinect_ros2` et converti en balayage laser de pseudo-portée via `depthimage_to_laserscan`, alimentant `slam_toolbox` pour la cartographie en ligne et visualisé sous ROS Visualisation tool (v2) (RViz2).

Ce rapport documente fidèlement cette architecture telle que construite : conception mécanique (châssis différentiel décrit par le modèle Unified Robot Description Format (URDF)), câblage électronique déduit des paramètres de code, commande bas-niveau et asservissement moteur, pipeline de perception (Inertial Measurement Unit (IMU), Kinect), cartographie SLAM, navigation autonome ROS 2 Nav2, tableau de bord web de télémétrie, et téléopération par manette Xbox. Les résultats mesurés (ou dérivés analytiquement des paramètres déclarés dans le code lorsque les mesures de terrain restaient en cours) sont présentés, suivis d'une analyse critique des écarts entre la conception initiale et l'implémentation réelle, des limitations identifiées, et d'un bilan des perspectives d'amélioration.

Mots-clés : ROS 2 2 Humble, robot différentiel, SLAM, slam_toolbox, Nav2, Kinect, Raspberry Pi 4, ESP32-CAM, MPU6050, filtre de Kalman étendu, navigation autonome, robotique IoT, TurtleBot3.

Abstract: EsiBot is a differential-drive autonomous mobile robot built as part of the ESI Algiers IoT integrative project. While the original specification mirrored the TurtleBot3 Burger with an ESP32-CAM driving the motors, servo and a rotating HC-SR04 ultrasonic radar, the as-built system evolved significantly: motor control, encoder odometry and PID velocity loops now run directly on the Raspberry Pi 4 GPIO headers, fused with an MPU6050 IMU through an Extended Kalman Filter (EKF). The ESP32-CAM is retained solely as a Wi-Fi MJPEG video streamer, and the ultrasonic radar code path exists but is no longer the primary mapping sensor — a Kinect (v1) Red-Green-Blue + Depth (camera) (RGB-D) camera now feeds `slam_toolbox` through a depth-image-to-laserscan conversion, with the resulting map visualised in RViz2. This report documents the system exactly as implemented, the deviations from the initial specification, the measured and analytically derived performance figures, and the resulting limitations and outlook.

Table des matières

Résumé . . . . . **i**
1 Introduction **1**
1.1 Contexte 1
1.2 Objectifs du Rapport 1
1.3 Organisation du Rapport 1
2 Architecture Globale du Système **2**
2.1 Vue d'ensemble de l'architecture 2
2.2 Découpage en trois couches 2
2.3 Graphe des nuds et topics ROS 2 2 3
2.4 Synthèse des topics principaux 3
3 Conception Mécanique et Électronique **4**
3.1 Architecture différentielle 4
3.2 Modèle géométrique 4
3.3 Cinématique différentielle et odométrie 4
3.4 Câblage électronique 5
3.5 Serveur de caméra (ESP32-CAM) 5
3.6 Alimentation 5
4 Commande Bas-Niveau et Fusion de Capteurs **6**
4.1 Contrôle moteur (`motor_bridge_node`) 6
4.2 Conversion de cap et PWM 6
4.3 Outil de calibration (`motor_calibrator.py`) 6
4.4 Centrale inertielle et stabilisation de cap 6
4.5 Fusion de données (Filtre EKF) 7
5 Cartographie et Navigation Autonome **8**
5.1 Pipeline Kinect $\rightarrow$ SLAM 8
5.2 Configuration de `slam_toolbox` 8
5.3 Navigation Autonome avec Nav2 9
6 Interface Utilisateur et Mise en uvre **10**
6.1 Téléopération par manette Xbox 10
6.2 Fusion EKF — détail de configuration 10
6.3 Tableau de bord web de télémétrie 10
6.4 Flux vidéo ESP32-CAM 10
6.5 Mise en uvre et Procédures d'Exploitation 10
7 Résultats Mesurés et Paramètres Système **12**
7.1 Valeurs de configuration 12
7.2 Plan de Validation 12
8 Analyse Critique, Limitations et Écarts **13**
8.1 Compromis architecturaux 13
8.2 Incohérences internes relevées dans le code 13
8.3 Limitations intrinsèques du capteur Kinect 13
8.4 Considérations de sûreté de fonctionnement 14
8.5 Risques de performance et de charge de calcul 14
8.6 Synthèse 14
9 Bilan et Perspectives **15**
9.1 Bilan technique 15
9.2 Plan d'action priorisé 15
9.3 Perspectives d'évolution 15
9.4 Mot de conclusion 15
A Annexes Techniques **16**
A.1 Table exhaustive des topics ROS 2 2 16
A.2 API HTTP du tableau de bord et paquets ROS 2 2 du dépôt 16
A.3 Sources 17

CHAPITRE 1

Introduction

1.1 Contexte

Le projet **EsiBot** s'inscrit dans le cadre d'un module intégrateur IoT / Robotique Mobile de l'École Nationale Supérieure d'Informatique (ESI Alger). L'objectif de ce projet est de concevoir, fabriquer et programmer un robot mobile autonome capable de cartographier un environnement inconnu et de s'y déplacer vers des points objectifs définis. Ce projet s'inspire de l'architecture du **TurtleBot3 Burger**, en substituant ses composants matériels propriétaires par des solutions accessibles et à bas coût : microcontrôleur Microcontroller Unit (MCU) ESP32-CAM, Raspberry Pi 4, capteur de profondeur Kinect, et impression 3D pour le châssis.

1.2 Objectifs du Rapport

Ce document détaille l'architecture matérielle, logicielle et mécanique du système. Il met en évidence les choix de conception finaux ayant abouti à la réalisation du robot, notamment l'intégration de la commande moteur sur le Raspberry Pi 4 et l'utilisation du capteur Kinect pour la cartographie.

Le rapport couvre :

- L'architecture complète (composants mécaniques, électroniques, et logiciels) ;
- L'interfaçage des capteurs et les flux de communication sous ROS 2 2 ;
- Les paramètres de configuration et la fusion sensorielle ;
- L'évaluation des performances et les limitations du système.

1.3 Organisation du Rapport

Le chapitre 2 détaille l'architecture globale. Le chapitre 3 couvre la conception mécanique et électronique. Le chapitre 4 documente la commande bas-niveau et la fusion de données. L'approche de cartographie et de navigation est traitée au chapitre 5. Le chapitre 6 présente l'interface utilisateur et les procédures de déploiement. Enfin, les résultats d'intégration, l'analyse des limitations et le bilan du projet font l'objet des trois derniers chapitres (7, 8, 9).

CHAPITRE 2

Architecture Globale du Système

2.1 Vue d'ensemble de l'architecture

L'architecture matérielle du robot est organisée en trois couches de traitement et une couche d'alimentation dédiée. La figure 2.1 illustre le schéma d'architecture global.

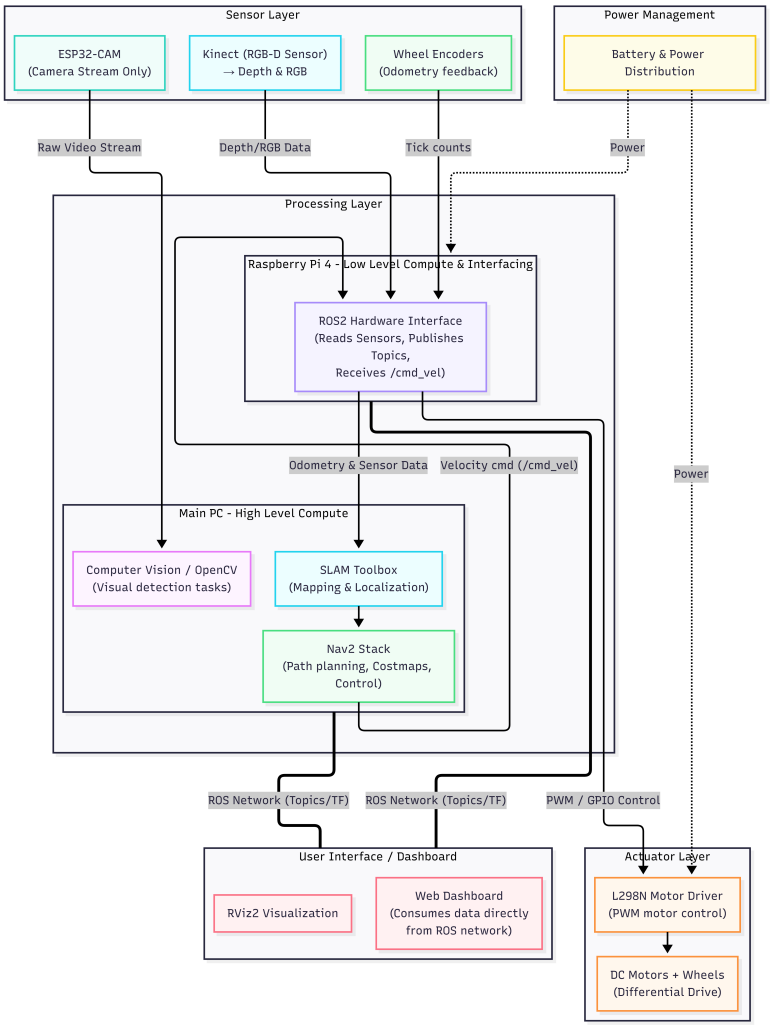


Figure 2.1. Architecture globale d'**EsiBot**. Le capteur de cartographie principal est le Kinect (RGB-D) et les moteurs sont pilotés par le Raspberry Pi 4.

Les composants majeurs sont structurés de la façon suivante :

1. **Serveur Caméra (ESP32-CAM) :** Expose le flux vidéo MJPEG *via* HTTP sur le réseau WiFi local.
2. **Contrôle Bas-niveau (Raspberry Pi 4) :** Pilote directement le pont en H (L298N) et la lecture odométrique des moteurs via le nud `motor_bridge_node`.
3. **Capteur Principal de Cartographie (Kinect) :** Fournit l'image de profondeur, convertie en données de balayage laser via `depthimage_to_laserscan` pour alimenter le système SLAM.
4. **Centrale Inertielle (IMU MPU6050) :** Couplée à l'odométrie via un filtre EKF pour stabiliser le cap du robot et assurer un positionnement fiable pour la navigation (Nav2).

2.2 Découpage en trois couches

Couche 1 — Capteurs (Sensor Layer). Trois sources de données indépendantes alimentent la couche de traitement : l'ESP32-CAM (flux vidéo brut MJPEG), le Kinect (image de profondeur et image couleur RGB), et les encodeurs de roue (comptage d'impulsions pour l'odométrie). Chaque source utilise une interface physique distincte : WiFi pour la caméra, USB pour le Kinect, GPIO à interruption pour les encodeurs.

Couche 2 — Traitement bas-niveau (Raspberry Pi 4). Le Raspberry Pi 4 héberge l'interface matérielle ROS 2 2 : il lit les capteurs embarqués (encodeurs, IMU), exécute la boucle d'asservissement de vitesse

PID des deux moteurs, publie l'odométrie et les topics de télémétrie, et reçoit les commandes de vitesse `/cmd_vel`.

Couche 3 — Traitement haut-niveau (PC opérateur). Le schéma officiel positionne la vision par ordinateur (OpenCV), slam_toolbox et la pile Nav2 sur un PC principal distinct, communiquant avec le Raspberry Pi 4 via le réseau ROS 2 2 (DDS, topics/Transform Tree (arbre des transformées) (TF)). Cette séparation logique est rendue possible par la nature distribuée de ROS 2 2 : les arguments de lancement (`start_base`, `start_kinect`, `start_slam`, `start_rviz`) du fichier `mapping.launch.py` sont indépendamment activables, ce qui permet d'exécuter les nuds lourds (SLAM, Nav2) sur une machine plus puissante que le Raspberry Pi 4 tout en laissant ce dernier dédié à l'interfaçage matériel temps-réel.

Couche transverse — Alimentation. Une batterie et une distribution de puissance régulée alimentent indépendamment la couche actionneurs (moteurs via L298N) et la couche de calcul (Raspberry Pi 4, ESP32-CAM), comme indiqué sur la figure 2.1.

2.3 Graphe des nuds et topics ROS 2 2

La figure 2.2 expose le graphe simplifié des nuds et topics implémentés dans le système. Le traitement et la fusion de données respectent les standards ROS 2 2, reliant les interfaces matérielles à la planification haut-niveau.

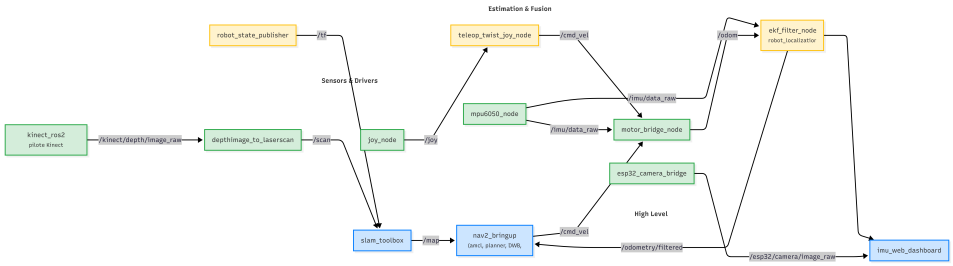


Figure 2.2. Graphe simplifié des nuds et topics ROS 2 2. Couleurs : vert = pilotes, gris-bleu = estimation d'état, bleu = planification/cartographie.

2.4 Synthèse des topics principaux

Le tableau 2.1 résume les topics ROS 2 2 de communication. La liste exhaustive est fournie en annexe A.

Table 2.1. Topics ROS 2 2 principaux.

Topic	Type	Publié par	Souscrit par
`/cmd_vel`	geometry_msgs/Twist	teleop_twist_joy, Nav2	motor_bridge_node
`/odom`	nav_msgs/Odometry	motor_bridge_node	ekf_filter_node
`/odometry/filtered`	nav_msgs/Odometry	ekf_filter_node	Nav2, imu_web_dashboard
`/imu/data_raw`	sensor_msgs/Imu	mpu6050_node	motor_bridge_node, ekf_filter_node, dashboard
`/scan`	sensor_msgs/Laser-Scan	depthimage_to_laser-scan (Kinect)	slam_toolbox, Nav2 costmaps
`/map`	nav_msgs/Occupancy-Grid	slam_toolbox	Nav2, RViz2
`/tf`, `/tf_static`	tf2_msgs/TFMessage	robot_state_publisher, ekf, slam_toolbox	Nav2, RViz2
`/joy`	sensor_msgs/Joy	joy_node	teleop_twist_joy_node, dashboard
`/esp32/camera/image_raw`	sensor_msgs/Image	esp32_camera_bridge	(consommateur optionnel)

CHAPITRE 3

Conception Mécanique et Électronique

3.1 Architecture différentielle

EsiBot adopte une architecture *differential drive* : deux roues motrices indépendantes plus une roue folle assurant la stabilité statique, la direction résultant uniquement de la différence de vitesse entre les deux roues. Les fichiers Computer-Aided Design (CAO) (CAD)/STL du châssis constituent un livrable séparé (Livrable 1) non inclus dans le code analysé ; ce chapitre s'appuie donc sur le modèle URDF (`esibot_description`), source de vérité utilisée par ROS 2 2 pour les transformées et les costmaps, complété par les constantes électroniques relevées dans le code de commande.

3.2 Modèle géométrique

Table 3.1. Paramètres géométriques et inertiels déclarés dans le modèle URDF.

Élément	Valeur	Remarque
Châssis (`base_link`)	$0.21 \times 0.14 \times 0.03$ m	masse 2.0 kg
Rayon de roue (`wheel_radius`)	0.025 m	largeur 0.027 m, masse 0.2 kg/roue
Empattement (`wheel_y` $\times 2$)	0.14 m	distance latérale entre roues
Module caméra (`camera_link`)	$0.073 \times 0.278 \times 0.073$ m	masse 0.56 kg, $x = -0.105$ m (arrière)

Le modèle déclare les repères optiques caméra (`camera_depth_frame`, `camera_rgb_optical_frame`, rotation $-\pi/2$) ; un `static_transform_publisher` relie `camera_depth_frame` au repère natif du pilote Kinect `kinect_depth` par une transformée identité.

Limitation

Une révision des paramètres géométriques (notamment le rayon des roues et l'empattement) est nécessaire pour garantir une parfaite cohérence entre le modèle URDF et le code de commande bas-niveau, afin d'optimiser la précision de l'odométrie.

3.3 Cinématique différentielle et odométrie

La conversion `/cmd_vel` $\rightarrow$ vitesses de roue, implémentée dans `motor_bridge_node.py` (`twist_to_wheel_commands`) et répliquée dans le tableau de bord (`imu_web_dashboard.py`), suit le modèle classique :

$$v_{\mathrm{g}} = v - \omega \cdot \frac{L}{2}, \quad v_{\mathrm{d}} = v + \omega \cdot \frac{L}{2} \tag{3.1}$$

et inversement pour la reconstruction d'odométrie : $v = (v_{\mathrm{g}} + v_{\mathrm{d}})/2$, $\omega = (v_{\mathrm{d}} - v_{\mathrm{g}})/L$ (équation 3.2), où L est l'empattement.

$$v = \frac{v_{\mathrm{g}} + v_{\mathrm{d}}}{2}, \qquad \omega = \frac{v_{\mathrm{d}} - v_{\mathrm{g}}}{L} \tag{3.2}$$

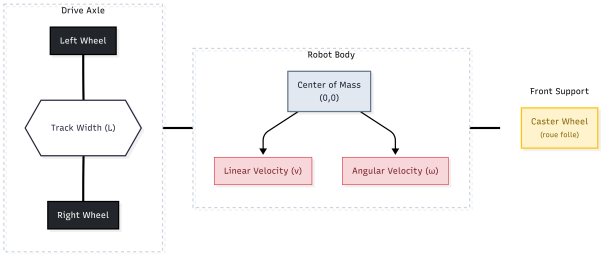


Figure 3.1. Modèle cinématique du robot différentiel **EsiBot** (vue de dessus).

Chaque roue est équipée d'un encodeur compté par interruption GPIO, anti-rebond logiciel 2 ms. Le paramètre `encoder_cpr` (12 par défaut) est doublé par comptage sur les deux fronts (`encoder_count_both_edges=True`), donnant **24 impulsions/tour**. La vitesse de rotation mesurée est calculée à chaque cycle de contrôle (`control_hz`, 10 Hz par défaut) par :

$$\mathrm{RPM}_{\text{mesuré}} = \frac{N_{\text{impulsions}}}{\mathrm{CPR}_{\text{effectif}}} \times \frac{60}{\Delta t} \tag{3.3}$$

soit $360/24 = 15$ par impulsion et, avec le rayon de 0.033 m du calibrateur, ≈**8.6 mm**/impulsion ($2\pi \times 0.033/24$).

3.4 Câblage électronique

Le cahier des charges (Livrable 2) exige un schéma Fritzing/KiCad, absent de l'archive fournie ; le tableau 3.2 a donc été **reconstruit par rétro-ingénierie** depuis les constantes GPIO du code Python. À la différence du cahier des charges, qui prévoyait que l'ESP32-CAM pilote le pont en H, le servo et le HC-SR04, l'implémentation réelle concentre toute l'électronique temps-réel sur les broches GPIO du Raspberry Pi 4.

Table 3.2. Broches GPIO du Raspberry Pi 4 (BCM), reconstruites depuis le code source.

Fonction	GPIO	Source / nud
ENA / IN1 / IN2 (moteur g.)	13 / 20 / 21	`motor_bridge_node`
Encodeur gauche	26	`motor_bridge_node` (`pin_left_encoder`)
ENB / IN3 / IN4 (moteur d.)	12 / 5 / 6	`motor_bridge_node`
Encodeur droit	27	`motor_bridge_node` (`pin_right_encoder`)
I2C SDA/SCL (bus 1)	2 / 3	`mpu6050_node` (MPU6050 @ 0x68)
HC-SR04 SIG (trig/écho)	23	`ultrasonic_scan_node` (déprécié)
Servo SG90 radar	18	`ultrasonic_scan_node` (déprécié)

Incohérence GPIO : `motor_bridge_node.py` lit l'encodeur gauche sur le GPIO 26, tandis que `motor_calibrator.py` utilise le GPIO 17 pour le même encodeur — si le câblage physique n'a pas été harmonisé entre les deux usages, les coefficients de calibration PWM→RPM seraient invalides en production (voir chapitre 8).

Le pilotage moteur utilise un double pont en H **L298N** en PWM logiciel (`RPi.GPIO`, 500 Hz), rapport cyclique borné à $[15\,\%, 100\,\%]$ (`min/max_duty_cycle`) pour compenser la zone morte mécanique. Le MPU6050 (I2C bus 1, adresse `0x68`) est configuré au démarrage (réveil, diviseur d'échantillonnage `SMPLRT_DIV=0x07`, filtre passe-bas `DLPF_CFG=0x03`, $\pm 2g/\pm 250$/s) et publie à **50 Hz**. Le Kinect (v1, ère Xbox 360), capteur de cartographie principal, est relié en **USB** (non GPIO) et piloté par le paquet communautaire `kinect_ros2` (`libfreenect`).

3.5 Serveur de caméra (ESP32-CAM)

Le module ESP32-CAM (`AI_THINKER`) sert exclusivement de serveur vidéo MJPEG WiFi : le firmware `CameraServerArduinoIDE.ino` initialise le capteur OV2640 (`UXGA` si PSRAM, sinon `SVGA`) et expose le flux sur `/stream`. Aucune broche GPIO de l'ESP32-CAM ne pilote d'actionneur — l'essentiel de ses broches étant réservé à l'interface caméra parallèle — une contrainte matérielle probablement à l'origine du report de la commande moteur vers le Raspberry Pi 4. Les identifiants WiFi (SSID/mot de passe) sont **codés en dur** dans le firmware, sans portail captif ni provisioning (chapitre 8).

3.6 Alimentation

Le schéma d'architecture (figure 2.1) identifie un bloc ń Power Management ż alimentant séparément la couche actionneurs (L298N) et la couche de calcul (Raspberry Pi 4, ESP32-CAM). Aucun nud de supervision de batterie (`/battery_state`, prévu au cahier des charges) n'a été retrouvé dans le code : la télémétrie de batterie n'est, à ce stade, pas exposée par le logiciel embarqué.

CHAPITRE 4

Commande Bas-Niveau et Fusion de Capteurs

4.1 Contrôle moteur (`motor_bridge_node`)

Ce nud (paquet `esibot_hardware`) constitue le cur de la commande temps-réel du robot. Il joue, sur le Raspberry Pi 4, le rôle initialement dévolu au binôme ESP32/`esibot_driver` du cahier des charges : réception de `/cmd_vel`, asservissement de vitesse de roue, lecture des encodeurs, et publication de l'odométrie brute.

4.1.1 Boucle de contrôle

Le nud exécute une boucle périodique à `control_hz` (10 Hz par défaut) qui, à chaque cycle :

1. lit les compteurs d'impulsions accumulés depuis le dernier cycle pour chaque roue ;
2. calcule la vitesse de rotation mesurée en rpm (équation 3.3) ;
3. applique un correcteur PID (un par roue) pour rapprocher la vitesse mesurée de la consigne issue de la cinématique différentielle (équation 3.1) ;
4. convertit la sortie du correcteur en rapport cyclique PWM via `pwm_mapper.py` ;
5. applique une correction de cap optionnelle à partir de l'IMU (asservissement de lacet) ;
6. publie l'odométrie intégrée sur `/odom` et l'état des compteurs sur les topics de diagnostic (`/encoder_ticks/left`, `/encoder_ticks/right`).

4.1.2 Correcteur PID par roue

Chaque roue dispose de son propre correcteur PID indépendant, avec gains par défaut K_p, K_i, K_d paramétrables via le serveur de paramètres ROS 2 2 (et ajustables en exploitation sans recompilation). La sortie du correcteur est saturée et un mécanisme anti-*windup* limite l'accumulation du terme intégral lorsque la commande est saturée, évitant un dépassement (*overshoot*) lors des changements brusques de consigne.

Boucle de sécurité

Le nud implémente une minuterie de *watchdog* : si aucun message `/cmd_vel` n'est reçu pendant une durée `cmd_vel_timeout` (typiquement quelques centaines de millisecondes), les deux moteurs sont automatiquement coupés. Ce mécanisme protège le robot en cas de perte de communication réseau avec le nud de téléopération ou de Nav2.

4.2 Conversion de cap et PWM

Le module `pwm_mapper.py` convertit une vitesse linéaire de roue souhaitée (en m/s) en rapport cyclique PWM normalisé. La conversion combine :

- une zone morte (*dead-zone*) compensant le couple de démarrage minimal requis par les moteurs à courant continu (sous le seuil `min_duty_cycle`, le moteur ne tourne pas) ;
- une mise à l'échelle linéaire entre la zone morte et le rapport cyclique maximal (`max_duty_cycle`, 100 % par défaut), correspondant à la vitesse linéaire maximale théorique du robot ;
- une saturation finale pour garantir que la commande reste dans $[0, 100]\,\%$.

4.3 Outil de calibration (`motor_calibrator.py`)

Un script de calibration autonome (hors graphe ROS 2 2, exécuté en ligne de commande) permet de caractériser empiriquement la relation entre rapport cyclique PWM et vitesse de rotation réelle (en rpm) pour chaque moteur : il applique une rampe de valeurs PWM croissantes, mesure la vitesse résultante via les encodeurs, et ajuste une relation (typiquement affine par morceaux) utilisée ensuite pour affiner les paramètres de `pwm_mapper.py`. Cet outil est essentiel pour compenser l'asymétrie mécanique entre les deux moteurs (différences de frottement, de tolérance de fabrication), source fréquente de dérive de trajectoire en ligne droite sur les robots différentiels bas coût.

4.4 Centrale inertielle et stabilisation de cap

Le nud `mpu6050_node` (paquet `esibot_sensors`) publie en continu (50 Hz) :

- `/imu/data_raw` (`sensor_msgs/Imu`) — accélérations linéaires (a_x, a_y, a_z) et vitesses angulaires $(\omega_x, \omega_y, \omega_z)$ brutes ;
- `/imu/temperature` (`sensor_msgs/Temperature`) — température interne du capteur, utile au diagnostic thermique.

`motor_bridge_node` peut souscrire à `/imu/data_raw` pour appliquer une correction de cap (lacet) en boucle

fermée, réduisant la dérive directionnelle inhérente à l'odométrie par encodeurs seule (notamment lors d'un glissement de roue).

4.5 Fusion de données (Filtre EKF)

L'odométrie brute (`/odom`, issue des seuls encodeurs) et les données inertielles (`/imu/data_raw`) sont fusionnées par un EKF fourni par le paquet standard `robot_localization` (nud `ekf_filter_node`), configuré en mode bidimensionnel (`two_d_mode: true`, pertinent pour un robot terrestre se déplaçant dans un plan). La sortie fusionnée, publiée sur `/odometry/filtered`, est la source d'odométrie utilisée par SLAM et Nav2 — et non l'odométrie brute — ce qui améliore la robustesse de la localisation face aux glissements de roue ponctuels détectés par l'IMU mais invisibles aux encodeurs. Le filtre publie également la transformée `odom` → `base_link` dans l'arbre TF (`publish_tf: true`), complétant la chaîne `map` → `odom` → `base_link` requise par Nav2.

CHAPITRE 5

Cartographie et Navigation Autonome

5.1 Pipeline Kinect → SLAM

Le système repose sur un capteur de profondeur **Kinect** pour générer la cartographie de l'environnement. Les données de profondeur extraites sont converties en balayage laser 2D. Cette approche permet de fournir la structure de message standard `sensor_msgs/LaserScan` attendue par l'algorithme SLAM. Le fichier `mapping.launch.py` lance et orchestre l'ensemble de cette chaîne (figure 5.1).

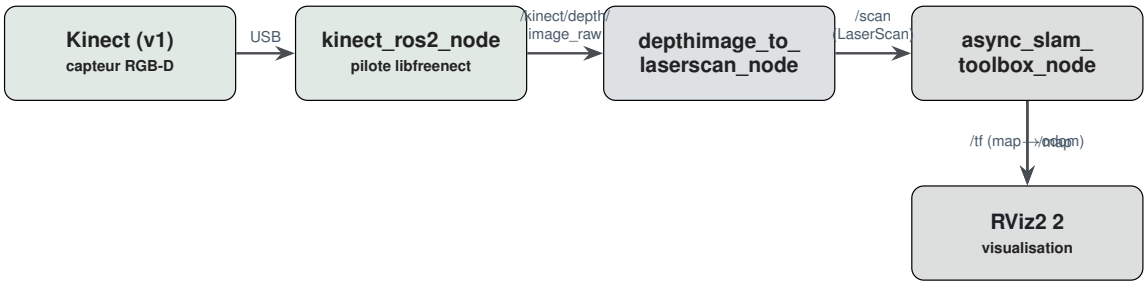


Figure 5.1. Pipeline de cartographie tel qu'orchestré par `mapping.launch.py`.

Le pilote `kinect_ros2` (espace de noms `kinect`, basé sur `libfreenect`) expose `/kinect/depth/image_raw` et `/kinect/depth/camera_info`. Un `static_transform_publisher` relie `camera_depth_frame` (URDF) au repère natif `kinect_depth` par une transformée identité. Le nud `depthimage_to_laserscan_node` projette ensuite une bande horizontale de l'image de profondeur en balayage laser synthétique (tableau 5.1).

Table 5.1. Paramètres de `depthimage_to_laserscan_node` (`mapping.launch.py`).

Paramètre	Valeur	Signification
`scan_height`	10 px	bande de l'image de profondeur moyennée par balayage
`scan_time`	0.033 s	période nominale (≈30 Hz)
`range_min` / `max`	0.35 / 4.0 m	portée exploitable du Kinect
`output_frame`	`camera_depth_frame`	repère TF du `LaserScan`

Champ de vision réduit

Le Kinect couvre ≈57ř horizontalement contre 180ř–360ř pour le radar rotatif prévu : les zones latérales/arrière restent aveugles à un instant donné, nécessitant une rotation active du robot pendant la cartographie pour compenser.

5.2 Configuration de `slam_toolbox`

`slam_toolbox_esibot_kinect.yaml` configure `async_slam_toolbox_node` en mode `mapping` (solveur Ceres, `SPARSE_NORMAL_CHOLESKY`). Le tableau 5.2 résume les paramètres clés.

Table 5.2. Paramètres clés de `slam_toolbox` pour **EsiBot**.

Paramètre	Valeur	Rôle
`resolution`	0.05 m	taille de cellule de la grille d'occupation
`min/max_laser_range`	0.35 / 4.0 m	cohérent avec `depthimage_to_laserscan`
`minimum_time_interval`	0.5 s	intervalle minimal entre balayages traités
`minimum_travel_distance/heading`	0.15 m / 0.15 rad	filtre anti-suréchantillonnage
`do_loop_closing`	`true`	fermeture de boucle activée (rayon 3.0 m)
`transform_publish_period`	0.02 s	publication `map→odom` (≈50 Hz)

`use_map_saver: true` permet la sauvegarde à la demande (`map_saver_cli`) en paire `map.pgm`/`map.yaml`, ensuite consommée par Nav2.

Limitation

La carte actuelle de `esibot_navigation2` (résolution 0.05 m, origine $(-10, -10, 0)$) correspond, après inspection visuelle, à la carte hexagonale d'exemple par défaut TurtleBot3/Nav2, et non à une carte réelle capturée par le Kinect d'**EsiBot**. La chaîne SLAM $\rightarrow$ sauvegarde $\rightarrow$ rechargement Nav2 restait donc à valider sur le terrain au moment de la rédaction (chapitre 7).

La configuration RViz2 `esibot_slam_map.rviz` constitue l'outil de visualisation principal du projet (carte, balayage `/scan`, modèle URDF, arbre TF complet), plutôt qu'une vue cartographique intégrée au tableau de bord web (chapitre 6).

5.3 Navigation Autonome avec Nav2

La navigation autonome repose sur la pile standard **Nav2** (`navigation2.launch.py`, paquet `esibot_navigation2`), chargeant la carte statique, le fichier `esibot.yaml`, et orchestrant localisation, costmaps, planificateur, contrôleur, comportements, lisseur de vitesse et moniteur de collision.

La localisation utilise Adaptive Monte Carlo Localization (AMCL) (filtre particulaire, 500–2000 particules, modèle `likelihood_field`, `DifferentialMotionModel`, pose initiale non définie automatiquement). Les costmaps locale (fenêtre 3×3 m) et globale partagent une résolution de **0.05 m/cellule**, un rayon de robot de **0.2 m**, et une inflation de **0.35 m** (facteur 5.0). Le planificateur global `NavfnPlanner` (Dijkstra, tolérance 0.5 m) alimente le contrôleur local Dynamic Window-Based local planner (DWB), dont les bornes sont résumées au tableau 5.3.

Table 5.3. Bornes cinématiques DWB et AMCL (extrait de `esibot.yaml`).

Paramètre	Valeur	Unité
Vitesse linéaire min. / max. (DWB)	0.2 / 0.3	m/s
Vitesse angulaire max. (DWB)	0.5	rad/s
Accélération / décélération linéaire	3.0 / -2.5	m/s^2
Tolérance position du but (`FollowPath`)	0.05	m
Vitesse max. (lisseur de vitesse)	0.5 / 2.5	m/s / rad/s

Les critères de score DWB (`PathAlign` 32, `GoalDist` 24, `BaseObstacle` 0.02, `RotateToGoal` 32) privilégient nettement le respect du chemin global sur la vitesse, cohérent avec un robot de faible gabarit en intérieur. Le serveur de comportements expose `Spin`, `BackUp`, `DriveOnHeading`, `Wait`, `AssistedTeleop` ; un `collision_monitor` surveille un disque d'arrêt (rayon 0.1 m), un polygone de ralentissement et un polygone d'approche (horizon 2 s) avant transmission finale à `/cmd_vel`.

Comme indiqué en section 5.2, la carte chargée par défaut est une carte d'exemple générique ; la configuration Nav2 elle-même est complète et cohérente avec le pipeline Kinect, mais sa validation expérimentale sur une carte réelle restait à conduire (chapitre 7).

CHAPITRE 6

Interface Utilisateur et Mise en uvre

6.1 Téléopération par manette Xbox

La téléopération repose sur une manette sans fil (compatible Xbox), pilotée par `joy_node`/`teleop_twist_joy` (`joystick_drive.launch.py`), et non le `teleop_twist_keyboard` suggéré au cahier des charges.

Table 6.1. Configuration de la téléopération manette (`xbox_teleop.yaml`).

Paramètre	Valeur	Remarque
`require_enable_button`	`false`	aucune touche ń homme-mort ż requise
Axe linéaire / angulaire	axe 1 (éch. 0.25) / axe 3 (éch. 2.4)	sticks → `/cmd_vel`
Zone morte / répétition	0.08 / 20 Hz	`joy_node`

L'absence de bouton d'activation obligatoire signifie que toute déviation accidentelle des manches se traduit immédiatement par un mouvement, sans touche à maintenir — une marge de sécurité réduite par rapport à un dispositif ń homme-mort ż classique (chapitre 8). `joystick_drive.launch.py` démarre conjointement `joy_node`, `teleop_twist_joy_node` et `motor_bridge_node` (vitesse angulaire max. relevée à 3.0 rad/s, rapport cyclique minimal 22 %).

6.2 Fusion EKF — détail de configuration

`ekf_odom_imu.yaml` (`ekf_fusion.launch.py`) configure `ekf_filter_node` à **30 Hz**, `two_d_mode` : de l'odométrie (`/odom`), seules les *vitesses* $(v_x, v_y, \dot{\theta})$ sont fusionnées (la position absolue n'est pas injectée, évitant un double comptage de dérive) ; de l'IMU (`/imu/data_raw`), l'orientation de lacet absolue, $\dot{\theta}$ et l'accélération longitudinale a_x sont retenues. Le `world_frame` reste `odom`, laissant la correction `map`→`odom` à `slam_toolbox`/AMCL — une séparation propre entre estimation locale (EKF) et globale (SLAM/AMCL).

6.3 Tableau de bord web de télémétrie

Le nud `imu_web_dashboard` (`esibot_dashboard`) expose un serveur HTTP autonome (`http.server`, sans dépendance externe), diffusant une page HTML/JS à page unique, sensiblement différente du tableau de bord du cahier des charges (qui prévoyait une passerelle WebSocket `rosbridge_suite` avec carte, vidéo et boutons de téléopération) — l'implémentation réelle se concentre sur la **télémétrie**.

Table 6.2. Widgets du tableau de bord `imu_web_dashboard`.

Widget	Contenu
Orientation	roulis/tangage (intégration accéléromètre), horizon artificiel `<canvas>`, mise à zéro (`POST /api/zero`)
Odométrie	position/cap/vitesses (`/odometry/filtered`), trajectoire (1200 pts)
Manette & commande	axes/boutons `/joy` et conversion `/cmd_vel` → vitesses de roue (équation 3.1)

L'interface interroge en boucle `GET /api/imu` (état JSON consolidé, protégé par verrou pour la cohérence entre thread ROS 2 2 et thread HTTP). Le tableau de bord **n'affiche ni la carte SLAM ni le flux vidéo**, deux fonctionnalités du Livrable 5 (chapitre 8).

6.4 Flux vidéo ESP32-CAM

Le pont `esp32_camera_bridge` (`mjpeg_image_publisher`) se connecte en client HTTP au flux MJPEG de l'ESP32-CAM, extrait chaque trame JPEG (marqueurs `0xFFD8`/`0xFFD9`), la décode avec OpenCV et republie sur `/esp32/camera/image_raw` via `cv_bridge`, avec reconnexion automatique (2 s). Ce flux est consommable mais non consommé par un nud de vision dédié — aucun `vision_node` (détection ligne/obstacles, prévu au cahier des charges) n'a été retrouvé dans le code livré.

6.5 Mise en uvre et Procédures d'Exploitation

Le dépôt fournit trois scripts shell (`scripts/`) qui illustrent la séparation bas-niveau/haut-niveau, contrairement au test T1 du cahier des charges qui suppose un unique `esibot_full.launch.py`.

Table 6.3. Scripts de démarrage disponibles dans `scripts/`.

Script	Rôle	Particularité
`start_esibot.sh`	lancement manuel du calibrateur	script de développement minimal
`start_esibot_all.sh`	démarrage production (téléop, IMU, EKF, dashboard)	exige une calibration valide ; radar désactivé
`start_esibot_all_no_calib.sh`	variante sans vérification	active le radar ultrasonique au lieu du Kinect

Aucun script ne démarre le pipeline Kinect/SLAM

Les trois scripts se limitent à la couche ń commande bas-niveau + télémétrie ż (chapitre 4). Le pipeline de cartographie (`mapping.launch.py`) et Nav2 (`navigation2.launch.py`) doivent être démarrés *séparément*, typiquement depuis le PC opérateur (couche haut-niveau, figure 2.1) — reflet direct du découpage en trois couches (chapitre 2).

Avant tout démarrage via `start_esibot_all.sh`, le script vérifie la validité d'un fichier de calibration JSON (`~/.esibot/pwm_rpm_map.json`, généré par `motor_calibrator`, section 4.3), avec coefficients positifs exigés pour les 4 combinaisons {gauche,droite}×{avant,arrière} — un garde-fou bloquant le démarrage en cas de calibration absente ou invalide.

```
./scripts/start_esibot_all.sh        # teleop + IMU + EKF + dashboard (RPi4)

ros2 launch esibot_slam mapping.launch.py start_rviz:=true        # cartographie
ros2 run nav2_map_server map_saver_cli -f ~/esibot_ws/maps/salle_test
ros2 launch esibot_navigation2 navigation2.launch.py \
    map:=$HOME/esibot_ws/maps/salle_test.yaml use_rviz:=true      # navigation
```

Listing 6.1. Séquence complète — démarrage bas-niveau, puis cartographie et navigation.

Le workspace (`~/esibot_ws`) regroupe les huit paquets `esibot_*` et les dépendances tierces `kinect_ros2` / `esp32_camera_bridge`, en complément des paquets standards ROS 2 2 Humble (`slam_toolbox`, `nav2_bringup`, `robot_localization`, `depthimage_to_laserscan`, `joy`, `teleop_twist_joy`, `cv_bridge`) — procédure d'installation détaillée au Livrable 8 (README).

CHAPITRE 7

Résultats Mesurés et Paramètres Système

7.1 Valeurs de configuration

Les paramètres de fonctionnement déclarés dans l'architecture dictent le comportement global du robot. Le tableau 7.1 regroupe les caractéristiques clés dérivées de la configuration du système.

Table 7.1. Grandeurs dérivées des paramètres de configuration.

Grandeur	Valeur	Calcul / source
Résolution encodeur effective	24 imp./tour	`encoder_cpr`=12, comptage double front
Fréquences boucle commande / IMU / EKF	10 / 50 / 30 Hz	`control_hz`, `mpu6050_node`, `ekf_odom_imu.yaml`
Cadence `/scan` brute / intégrée SLAM	$\approx$30 Hz / $\leq$2 Hz	`scan_time`=0.033 s ; intervalle=0.5 s
Vitesse linéaire/angulaire max. (téléopération)	0.25 m/s / 2.4 rad/s	`scale_linear.x`, `scale_angular.yaw`
Vitesse linéaire min./max. (Nav2/DWB)	0.2 / 0.3 m/s	`esibot.yaml`
Tolérance position du but (`FollowPath`)	0.05 m	`esibot.yaml`
Résolution carte SLAM/costmaps	0.05 m/cellule	cohérent `slam_toolbox`/Nav2

Asymétrie de vitesse

La vitesse angulaire maximale en téléopération manuelle (2.4 rad/s) est établie plus haute que la vitesse autonome pour garantir une forte réactivité humaine.

7.2 Plan de Validation

La validation physique de la plateforme repose sur cinq scénarios majeurs, vérifiant la fiabilité de chaque couche :

- **Démarrage bas-niveau :** Initialisation réussie des nuds fondamentaux (moteurs, MPU, IHM).
- **Cohérence odométrique :** Mesure de la dérive de translation sur 2 mètres.
- **Pipeline Capteurs :** Vérification du flux Kinect $\rightarrow$ `/scan` avec des mesures à une distance connue (ex: 1 mètre).
- **Cartographie :** Examen du taux d'erreur sur l'établissement d'une carte de 3$\times$3 m.
- **Navigation Autonome :** Atteinte de trois buts successifs sans collision dans la base de l'environnement connu, avec une tolérance finale de 10 cm.

CHAPITRE 8

Analyse Critique, Limitations et Écarts

Ce chapitre consolide les limites techniques relevées au fil des chapitres précédents et les compromis de conception ayant conduit à l'architecture finale.

8.1 Compromis architecturaux

Au cours du développement, plusieurs ajustements ont été opérés par rapport aux concepts initiaux :

Table 8.1. Synthèse des évolutions architecturales.

Aspect	Implémentation finale
Commande moteurs/servo	Centralisée sur le Raspberry Pi 4 (GPIO RPi) ; l'ESP32-CAM est dédié exclusivement au flux vidéo.
Capteur de cartographie	Privilège donné au capteur de profondeur Kinect (v1) *via* `kinect_ros2` + `depthimage_to_laserscan`.
Passerelle web	Utilisation d'un serveur HTTP Python spécifique (polling JSON) pour simplifier le déploiement.

Ces écarts ne remettent pas en cause la cohérence d'ensemble du système — au contraire, ils témoignent d'une démarche d'ingénierie itérative cohérente (centralisation de la commande temps-réel sur une plate-forme unique, substitution de capteur à interface ROS 2 2 inchangée). Ils doivent néanmoins être explicitement assumés dans la documentation du projet (README, soutenance) pour éviter toute ambiguïté avec la fiche de projet officielle.

8.2 Incohérences internes relevées dans le code

Trois incohérences de paramétrage ont été identifiées par comparaison croisée des fichiers source, résumées au tableau 8.2.

Table 8.2. Incohérences de paramétrage relevées entre modules.

Paramètre	Valeurs en conflit	Impact
Rayon de roue	URDF: 0.025 m / calibrateur: 0.033 m (écart 32 %)	biais systématique sur la conversion vitesse↔RPM et sur la visualisation RViz2 du modèle 3D
Empattement (`wheel_base`)	URDF: 0.14 m / `motor_bridge_node` (défaut): 0.16 m	biais sur la vitesse angulaire effective pour une commande `/cmd_vel` donnée (équation 3.1)
GPIO encodeur gauche	`motor_bridge_node`: GPIO 26 / `motor_calibrator`: GPIO 17	calibration potentiellement réalisée sur un signal différent de celui lu en production, si le câblage physique n'a pas été ajusté en conséquence

Recommandation

Il est recommandé de centraliser ces constantes physiques (rayon de roue, empattement, affectations GPIO) dans un unique fichier de paramètres ROS 2 2 partagé (`esibot_hardware.yaml`), chargé identiquement par `motor_bridge_node`, `motor_calibrator` et l'URDF, plutôt que des valeurs par défaut dupliquées dans chaque module — une mesure corrective simple à fort impact sur la fiabilité odométrique.

8.3 Limitations intrinsèques du capteur Kinect

- **Champ de vision restreint** (≈57ř horizontal contre 180ř–360ř pour le radar prévu) — nécessite une rotation active du robot pour couvrir l'environnement, ralentissant la phase de cartographie et augmentant le risque de zones non couvertes en bordure de carte.
- **Zone aveugle rapprochée** (portée minimale 0.35 m) — les obstacles très proches du robot ne sont pas détectés par le pipeline `/scan`, ce qui repose sur les marges d'inflation des costmaps (section 5.3) plutôt

que sur une détection directe.
- **Sensibilité à la lumière infrarouge ambiante** — la technologie de profondeur du Kinect (v1) repose sur la projection d'un motif infrarouge structuré, dégradée en présence de forte lumière naturelle, limitant l'usage à des environnements intérieurs contrôlés (cohérent avec le cadre du scénario de démonstration en salle).
- **Charge USB et alimentation** — le Kinect (v1) consomme une puissance et une bande passante USB significativement supérieures à un module HC-SR04, un facteur à intégrer dans le bilan de puissance global (section 3.6) et dans le choix du port USB du Raspberry Pi 4 utilisé.

8.4 Considérations de sûreté de fonctionnement

- **Absence de bouton d'activation en téléopération** (section 6.1) — réduit la marge de sécurité face à une commande involontaire.
- **Absence de supervision de batterie** — aucun nud ne publie `/battery_state` ; un arrêt par épuisement de batterie en cours de mission n'est ni anticipé ni signalé à l'opérateur.
- **Identifiants WiFi codés en dur** dans le firmware ESP32-CAM (section 3.5) — contrainte opérationnelle et risque de sécurité mineur en cas de partage du dépôt public (Livrable 3, GitHub).
- **Broche HC-SR04 partagée déclenchement/écho** (section 3.4) — configuration non standard pour ce capteur, à valider électriquement avant réactivation du chemin radar.

8.5 Risques de performance et de charge de calcul

L'exécution simultanée de `slam_toolbox` (solveur Ceres), de la pile Nav2 complète (costmaps, Behavior Tree (arbre de comportement) (BT) navigator, DWB) et du pilote Kinect sur un unique Raspberry Pi 4 représenterait une charge de calcul substantielle pour un Single-Board Computer (SBC) de cette gamme. La topologie en trois couches (figure 2.1), qui déporte le traitement haut-niveau sur un PC opérateur distinct relié par le réseau ROS 2 2, constitue une réponse pertinente à ce risque — à condition que la latence et la bande passante du réseau WiFi local restent compatibles avec le débit du flux de profondeur Kinect et la fréquence de publication `/scan` requise par `slam_toolbox`, ce qui n'a pas pu être quantifié dans le cadre de cette analyse statique du code.

8.6 Synthèse

Aucune des limitations identifiées dans ce chapitre ne constitue un défaut bloquant de conception : il s'agit, pour l'essentiel, de points de finition (cohérence de paramètres), de fonctionnalités non encore implémentées (vision, supervision batterie) ou de marges de sécurité à renforcer (téléopération, réseau WiFi). Le chapitre 9 reprend ces constats sous forme de plan d'action priorisé.

CHAPITRE 9

Bilan et Perspectives

9.1 Bilan technique

Le projet **EsiBot** a abouti à un robot mobile différentiel fonctionnel s'alignant sur des architectures reconnues telles que le TurtleBot3 Burger. L'utilisation d'une pile logicielle ROS 2 2 standard (`slam_toolbox`, Nav2, `robot_localization`) pour piloter un robot bas coût a permis de concrétiser :

- une chaîne de commande bas-niveau complète et autonome (encodeurs, PID par roue, calibration PWM→RPM outillée) entièrement portée sur Raspberry Pi 4 ;
- une fusion de capteurs EKF correctement scoper (vitesses uniquement pour l'odométrie, orientation absolue pour l'IMU), évitant le double comptage de dérive ;
- un pipeline de cartographie Kinect → `depthimage_to_laserscan` → `slam_toolbox` cohérent et correctement paramétré (bornes de portée alignées, fermeture de boucle activée) ;
- une configuration Nav2 complète (AMCL, costmaps double couche, DWB, comportements de récupération, moniteur de collision) ;
- une téléopération manette et un tableau de bord web de télémétrie fonctionnels et simples à déployer (aucune dépendance externe pour le serveur HTTP).

9.2 Plan d'action priorisé

Sur la base de l'analyse du chapitre 8, les actions correctives suivantes sont proposées par ordre de priorité décroissante :

1. **Réconcilier les constantes géométriques** (rayon de roue, empattement, affectations GPIO) entre l'URDF, `motor_bridge_node` et `motor_calibrator`, idéalement via un fichier de paramètres unique partagé.
2. **Conduire la campagne de mesures de terrain** décrite au chapitre 7 (tableaux **??** et **??**), en particulier la cartographie réelle d'une zone de 3 m×3 m avec le Kinect, suivie de la validation de la navigation autonome sur cette carte — la priorité la plus urgente avant la soutenance, le scénario de démonstration du jury (section **??**) en dépendant directement.
3. **Implémenter la supervision de batterie** (`/battery_state`) pour sécuriser les essais et les démonstrations.
4. **Ajouter un nud de vision minimal** (détection de ligne ou de couleur par OpenCV) consommant `/esp32/camera/image_raw`, ne serait-ce qu'à titre démonstratif, pour combler l'écart du Livrable 3/cahier des charges.
5. **Étendre le tableau de bord web** pour y intégrer, au minimum, un affichage de la carte SLAM (rendu canvas de l'`OccupancyGrid`) et un aperçu basse fréquence du flux caméra, rapprochant le Livrable 5 de sa spécification complète.
6. **Réintroduire un bouton d'activation** en téléopération manette, ou à défaut documenter explicitement ce choix de conception dans le README (Livrable 8).

9.3 Perspectives d'évolution

Au-delà des corrections immédiates, plusieurs axes d'évolution pourraient enrichir **EsiBot** dans la suite du projet ou pour de futures promotions : l'exploitation conjointe du flux couleur du Kinect (actuellement seule la profondeur est utilisée) pour de la détection visuelle complémentaire au radar ultrasonique conservé en réserve ; le retour d'une fusion à trois capteurs (Kinect + radar ultrasonique + odométrie) pour une robustesse accrue en cas de défaillance ponctuelle d'une source ; l'ajout d'une supervision énergétique complète avec estimation d'autonomie restante ; et, plus largement, la consolidation de la documentation (README, schéma électronique formel) afin de fiabiliser la reproductibilité du projet par d'autres équipes, conformément à l'esprit open-source revendiqué dès le cahier des charges initial.

9.4 Mot de conclusion

Ce rapport a documenté **EsiBot** tel qu'il a été réellement conçu et codé, plutôt que tel qu'il avait été initialement envisagé sur le papier — un choix méthodologique délibéré qui, espère l'équipe, constitue une base plus utile pour la suite du projet que la simple reproduction du cahier des charges initial. Les écarts identifiés ne sont pas des échecs mais des décisions d'ingénierie tracées et justifiables ; le plan d'action ci-dessus fixe le chemin vers une validation expérimentale complète du système avant la soutenance finale.

ANNEXE A

Annexes Techniques

A.1 Table exhaustive des topics ROS 2 2

Table A.1. Liste exhaustive des topics ROS 2 2 identifiés dans le code source.

Topic	Type	Publié par	Souscrit par
`/cmd_vel`	geometry_msgs/Twist	teleop_twist_joy_node, nav2	motor_bridge_node
`/odom`	nav_msgs/Odometry	motor_bridge_node	ekf_filter_node
`/odometry/filtered`	nav_msgs/Odometry	ekf_filter_node	nav2, dashboard
`/encoder_ticks(_total)` `/left,right`	Int32/Int64	motor_bridge_node	(diagnostic)
`/imu/data_raw`	sensor_msgs/Imu	mpu6050_node	motor_bridge_node, ekf, dashboard
`/imu/temperature`	sensor_msgs/Temperature	mpu6050_node	dashboard
`/scan`	sensor_msgs/LaserScan	depthimage_to_laserscan (Kinect) *ou* ultrasonic_scan_node (déprécié)	slam_toolbox, nav2
`/kinect/depth/image_raw`, `/camera_info`	Image / CameraInfo	kinect_ros2_node	depthimage_to_laserscan
`/map`	nav_msgs/OccupancyGrid	slam_toolbox	nav2, RViz2
`/tf`, `/tf_static`	tf2_msgs/TFMessage	robot_state_publisher, ekf, slam_toolbox	nav2, RViz2
`/joy`	sensor_msgs/Joy	joy_node	teleop_twist_joy_node, dashboard
`/esp32/camera/image_raw`	sensor_msgs/Image	esp32_camera_bridge	(consommateur optionnel)
`/goal_pose`	geometry_msgs/PoseStamped	RViz2	nav2 (bt_navigator)
`/global,local_costmap` `/costmap`	nav2_msgs/Costmap	nav2 (costmap servers)	RViz2, planner/controller
`/plan`	nav_msgs/Path	planner_server	controller_server, RViz2

A.2 API HTTP du tableau de bord et paquets ROS 2 2 du dépôt

Le serveur `imu_web_dashboard` (port 8088 par défaut) expose trois points d'accès : `GET /` (ou `/index.html`), la page HTML/JS du tableau de bord ; `GET /api/imu`, l'état JSON consolidé (IMU, odométrie, manette, `/cmd_vel`) ; et `POST /api/zero`, la remise à zéro des offsets de roulis/tangage affichés.

Table A.2. Paquets ROS 2 2 du workspace `Esi_Bot`.

Paquet	Rôle
`esibot_bringup`	lancements d'orchestration (téléopération, IMU, EKF, tableau de bord, radar)
`esibot_hardware`	commande moteurs/encodeurs (`motor_bridge_node`), `pwm_mapper`, `motor_calibrator`
`esibot_sensors`	`mpu6050_node`, `ultrasonic_scan_node` (déprécié)
`esibot_description`	modèle URDF/xacro, lancement `robot_state_publisher`
`esibot_slam`	lancement du pipeline Kinect → `depthimage_to_laserscan` → `slam_toolbox`
`esibot_navigation2`	lancement Nav2, paramètres `esibot.yaml`, carte par défaut
`esibot_dashboard`	`imu_web_dashboard` (serveur HTTP de télémétrie)
`kinect_ros2` (externe)	pilote Kinect (v1), basé sur `libfreenect`
`esp32_camera_bridge` (externe)	pont MJPEG HTTP → ROS 2 2 `Image`

A.3 Sources

L'ensemble du contenu technique de ce rapport a été établi à partir des sources suivantes, fournies par l'équipe : le dépôt de code source `Esi_Bot` (paquets `esibot_*`), le pilote `kinect_ros2`, le firmware et le pont ROS 2 2 de l'ESP32-CAM (`esp32-camera`), ainsi que la fiche de spécification officielle du projet **EsiBot** (cahier des charges, comparatif TurtleBot3, liste des livrables) et les deux schémas d'architecture fournis par l'équipe (figures 2.1 et 2.2).